Credit Card Fraud Detection Using Multi-Layer Perceptrons and Focal Loss

Author: George-Cătălin Rece

Course: Advanced Artificial Intelligence and Machine Learning Research (AAIMR)

Coordinator: Prof. Dr. Costin Bădică

Assignment: Research Project #1 (Section 2.1: Multi-Layer Perceptron - Credit Card Fraud Detection)

Google Colab Notebook: https://colab.research.google.com/github/JeyJey9/credit-card-fraud-detection-dl/blob/main/Credit_Card_Fraud_Detection_DL.ipynb

GitHub Repository: https://github.com/JeyJey9/credit-card-fraud-detection-dl

Date: November 2026

Abstract

Automated detection of credit card fraud is an important application of machine learning in financial systems. From a modeling standpoint, the main difficulty is the severe class imbalance between normal and fraudulent transactions, combined with asymmetric error costs. In this report, we implement and evaluate a Multi-Layer Perceptron (MLP) pipeline for binary fraud classification on the European cardholders dataset published by the Machine Learning Group at Université Libre de Bruxelles (ULB) [33], which contains 284,807 transactions and 492 frauds (0.1727%). Starting from a baseline feed-forward network [1], we compare three loss formulations: standard Binary Cross-Entropy (BCE), cost-sensitive weighted BCE, and Binary Focal Loss [31] with prior bias initialization. In addition, we implement the three extensions proposed in the project specification: (1) post-hoc explainability using SHapley Additive exPlanations (SHAP) [32], (2) automated hyperparameter tuning using Optuna [5], and (3) a performance comparison against Gradient Boosted Decision Trees using XGBoost [7]. On the held-out test split (42,722 transactions, 74 frauds), the Optuna-tuned Focal Loss MLP (256-128-64 layers, SiLU activation, α=0.65, γ=1.75) reaches a validation PR-AUC of 0.8798 and achieves the highest calibrated test F1-score (0.8286, Precision = 87.88%, Recall = 78.38%, MCC = 0.8297), generating only 8 false positives across 42,648 normal transactions. Compared to XGBoost (test PR-AUC = 0.8422, test F1 = 0.8244, Precision = 94.74%, Recall = 72.97%), the tuned MLP identifies more fraudulent transactions on the test set (58 vs. 54 true positives), while XGBoost produces fewer false positives (3 vs. 8).

1. Introduction

1.1 Problem Context and Motivation

With the growth of online banking and cashless payments, card issuers process millions of transactions every hour. Detecting unauthorized transactions automatically is necessary because manual review can only cover a tiny fraction of daily traffic [6]. If a fraudulent transaction goes unnoticed, the bank or merchant absorbs the financial loss. Conversely, if legitimate transactions are mistakenly declined, customers are inconvenienced and support teams have to handle unnecessary alerts.

In machine learning terms, transaction fraud detection is framed as a supervised binary classification task. Each transaction is described by a feature vector $x_i \in R^d$ containing transaction details and historical aggregates, and the model predicts a probability $P(y_i = 1 \mid x_i)$, where $y_i = 1$ indicates fraud and $y_i = 0$ indicates a normal transaction [33].

1.2 Main Technical Challenges

Working with credit card fraud data involves several practical difficulties:

1. **Severe Class Imbalance:** Fraud represents a very small fraction of all transactions. In the ULB dataset [33], only 492 out of 284,807 records are positive (0.1727%, or roughly a 1 : 578 ratio). When training a neural network with standard cross-entropy, the gradient updates are dominated by the large number of easy negative examples.
2. **Misleading Standard Metrics:** Raw classification accuracy is not useful here, because a model that always predicts class 0 already reaches 99.83% accuracy. Even ROC-AUC can be overly optimistic when True Negatives are very large [13]. For this reason, Precision-Recall AUC (PR-AUC / Average Precision), F1-score, and the Matthews Correlation Coefficient (MCC) are much better indicators of actual performance.
3. **Precision-Recall Trade-off:** Lowering the classification threshold catches more frauds (higher recall) but also increases false alarms (lower precision). Choosing the right operating threshold on a validation set is therefore just as important as training the model itself [6].
4. **Model Interpretability:** Neural networks act as black-box predictors by default. In financial applications, analysts and auditors need to know which input features caused a transaction to be flagged as suspicious [32].

1.3 Objectives and Report Structure

While tree-based models such as XGBoost [7] are widely used on tabular datasets, Multi-Layer Perceptrons (MLPs) remain very flexible when combined with modern regularization and custom loss functions such as Focal Loss [31]. Starting from the baseline approach outlined by Gulhane [1], the goals of this project are:

- To build a well-regularized PyTorch MLP architecture using Batch Normalization [8], Dropout [9], and smooth activation functions (`GELU` [10], `SiLU` [11]).
- To evaluate how standard BCE, class-weighted BCE, and Binary Focal Loss [31] handle the 1:578 class imbalance.
- **Extension 1 (Explainable AI):** To explain the MLP predictions at both the global dataset level and the individual transaction level using SHAP [32].
- **Extension 2 (Hyperparameter Tuning):** To use Optuna [5] for automated tuning of the network architecture, regularization, learning rate, and Focal Loss parameters.
- **Extension 3 (Gradient Boosting Comparison):** To compare the neural models against an XGBoost baseline [7] and integrate the trained models into an interactive Streamlit web dashboard.

2. Deep Learning Methodology

2.1 Multi-Layer Perceptron (MLP) Architecture

A Multi-Layer Perceptron (MLP) processes a tabular input vector $x = h^{(0)} \in R^{d}$ ($d = 30$) through L fully connected hidden blocks followed by a single linear output unit. In our implementation, each hidden block $l \in \{1, ..., L\}$ applies a linear layer, 1D Batch Normalization, a non-linear activation function $\varphi(\cdot)$, and Dropout:

$$z^{(l)} = W^{(l)} h^{(l-1)} + b^{(l)}, \; W^{(l)} \in R^{d_l \times d_{l-1}}, \; b^{(l)} \in R^{d_l}$$

$$\hat{z}^{(l)} = \mathrm{BatchNorm1d}(z^{(l)}) = \gamma^{(l)} \odot \frac{z^{(l)} - \mu_B^{(l)}}{\sqrt{(\sigma_B)^{(l)})^2 + \varepsilon}} + \beta^{(l)}$$

$$h^{(l)} = \mathrm{Dropout}_p(\varphi(\hat{z}^{(l)}))$$

Batch Normalization [8] normalizes layer activations within each mini-batch B, which helps stabilize training when input features have heavy tails. Dropout [9] randomly deactivates a proportion p of hidden units during training so that the network does not overfit to the 344 positive fraud samples in the training set.

Alongside the standard ReLU(z) = max(0, z) activation, we also test two smooth activations during hyperparameter tuning:

- **GELU** [10]: GELU(z) = z · Φ(z) = (1) / (2) z [1 + erf((z) / (√(2)))].
- **SiLU (Swish)** [11]: SiLU(z) = z · σ(z) = (z) / (1 + e^{-z)}.

Because many PCA components take negative values for fraudulent transactions (for example `V14` and `V10`), smooth activations like `SiLU` allow small negative values and gradients to pass through rather than clipping them hard at zero.

The output layer computes a scalar logit z_{out} = w_{out}^T h^{(L)} + b_{out}, which is converted into a fraud probability through the sigmoid function:

$$hat(p) = P(y = 1 \mid x) = σ(z_{out}) = (1) / (1 + exp(-z_{out}))$$

2.2 Loss Functions for Imbalanced Classification

Standard Binary Cross-Entropy (BCE)

For a ground-truth label y ∈ {0, 1} and predicted probability hat(p) = σ(z_{out}), we denote the probability assigned to the true class by p_t:

$$p_t = \{ hat(p) \text{ if } y = 1 \text{ ; } 1 - hat(p) \text{ if } y = 0 \}$$

Standard BCE is given by L_{BCE}(p_t) = -log(p_t). When negative examples outnumber positive ones by 578 : 1, the sum of small losses from thousands of well-classified negative transactions (p_t > 0.99) can still outweigh the loss from the few positive fraud samples.

Cost-Sensitive Weighted BCE

A straightforward way to counter class imbalance is to multiply the positive class loss by a weight w_+ > 1:

$$L_{W\text{-}BCE}(y, hat(p)) = -w_+ y \log(hat(p)) - (1 - y) \log(1 - hat(p))$$

Using the exact inverse class ratio w_+ = N_{neg} / N_{pos} ≈ 578.5 often causes the network to over-predict the positive class and lowers precision. Instead, we use a moderated square-root weight w_+ = √(N_{neg) / N_{pos}} ≈ 24.05, which increases fraud sensitivity without completely destabilizing the predicted probabilities.

Binary Focal Loss and Prior Bias Initialization

Focal Loss was proposed by Lin et al. [31] to address severe class imbalance in one-stage object detectors. It modifies cross-entropy by adding a modulating factor (1 - p_t)^γ with focusing parameter γ ≥ 0 and a class balance factor α ∈ [0, 1]:

$$L_{Focal}(p_t) = -α_t (1 - p_t)^γ \log(p_t), \text{ where } α_t = \{ α \text{ if } y = 1 \text{ ; } 1 - α \text{ if } y = 0 \}$$

As shown in **Figure 2**, the factor (1 - p_t)^γ reduces the loss contribution of easy examples while keeping the loss high for misclassified examples:

1. If an example is misclassified and p_t is small, (1 - p_t)^γ stays close to 1, so the gradient is not reduced.
2. If an example is easily classified (p_t → 1), (1 - p_t)^γ goes to 0. For γ = 2, an easy negative with p_t = 0.99 has its weight scaled down by (1 - 0.99)^2 = 10^{-4}.

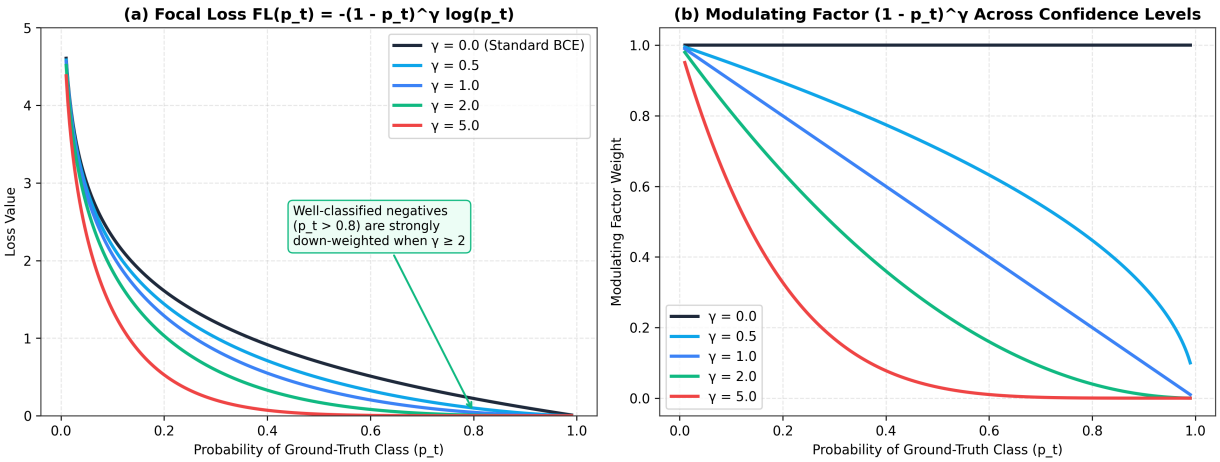


Figure 2: Theoretical Focal Loss curves and modulating factor across confidence levels.

We also apply the bias initialization trick suggested by Lin et al. [31]. If the output bias b_{out} starts at 0, the initial prediction is hat(p) ≈ 0.5, which produces huge losses on the 99.83% negative majority during the first few batches. Setting the initial bias using the prior fraud rate π ≈ 0.001725:

$$b_{out}^{(0)} = -log((1 - \pi) / (\pi)) \approx -6.36$$

ensures that initial predictions start close to the true fraud frequency.

2.3 Extension 1: Explainable AI with SHAP

To understand how the trained MLP reaches its decisions, we use **SHAP (SHapley Additive exPlanations)** [32]. Based on cooperative game theory, SHAP decomposes a model prediction f(x) into a baseline expectation φ_0 = E[f(X)] plus the marginal contribution φ_j(x) of each feature j ∈ {1, ..., d}:

$$f(x) = \varphi_0 + \sum_{j=1}^{d} \varphi_j(x)$$

$$\varphi_j(x) = \sum_{S \subseteq \{1, \ldots, d\} \setminus \{j\}} (|S|! \, (d - |S| - 1)!) / (d!) \, [f_x(S \cup \{j\}) - f_x(S)]$$

Because computing exact Shapley values over all 2^{30} subsets is too slow, we use `shap.GradientExplainer` (Expected Gradients) on our PyTorch network. This method estimates feature attributions by integrating gradients along straight paths between a background sample x' ~ D_{bg} and the input transaction x:

$$\varphi_j^{Grad}(x) = E_{x' \sim D_{bg}, u \sim U(0, 1)} [(x_j - x'_j) \times (\partial f(x' + u(x - x'))) / (\partial x_j)]$$

2.4 Extension 2: Automated Hyperparameter Tuning with Optuna

Instead of manually guessing the network width, dropout rate, and Focal Loss parameters, we use **Optuna** [5] with its default **Tree-structured Parzen Estimator (TPE)** sampler. TPE models the hyperparameter space by splitting completed trials into two density estimators: l(λ) for trials that achieved top validation PR-AUC scores, and g(λ) for the remaining trials. New candidates are sampled by maximizing the ratio l(λ) / g(λ). After the study finishes, we also run functional ANOVA (fANOVA) to measure which hyperparameters had the largest effect on validation PR-AUC.

2.5 Extension 3: Gradient Boosting Baseline (XGBoost)

To see how our deep learning models compare against a standard tabular method, we train an **XGBoost** classifier [7]. XGBoost builds an ensemble of decision trees hat(y)_i = ∑_{k=1}^{K} f_k(x_i) sequentially, where each new tree f_t minimizes a second-order Taylor approximation of the regularized log-loss:

$$tilde(L)^{(t)} = \sum_{i=1}^{N} [g_i f_t(x_i) + (1) / (2) h_i f_t^2(x_i)] + \gamma_{tree} T + (1) / (2) \lambda_{reg} \sum_{j=1}^{T} w_j^2$$

where g_i and h_i are the first and second derivatives of the loss with respect to the previous prediction.

3. Software Design and Implementation

3.1 Codebase Organization

We used the article by Gulhane [1] as a conceptual starting point, but wrote the experimental pipeline in Python 3.11 and PyTorch 2.15 as a modular project (`src/`). The full source code, computational notebook (`Credit_Card_Fraud_Detection_DL.ipynb`), trained model checkpoints, and presentation slides are hosted in the public GitHub repository at https://github.com/JeyJey9/credit-card-fraud-detection-dl (and can also be opened directly in Google Colab at https://colab.research.google.com/github/JeyJey9/credit-card-fraud-detection-dl/blob/main/Credit_Card_Fraud_Detection_DL.ipynb). Separating data loading, model definition, loss functions, and evaluation makes it easier to run fair comparisons and reuse the trained models in the Streamlit dashboard.

Module / File	Main Functions Classes	Role in the Project
`src/download_data.py`	`ensure_dataset()`	Downloads `creditcard.csv` if missing and checks that it has 284,807 rows, 31 columns, and 492 frauds.
`src/data_utils.py`	`set_seed()`, `load_and_preprocess_data()`, `create_dataloaders()`	Fixes random seeds (SEED=42), performs the stratified 70%/15%/15% split, fits `RobustScaler` on training `Time` and `Amount`, and builds PyTorch `DataLoader` objects.
`src/losses.py`	`BinaryFocalLoss(nn.Module)`	Implements Binary Focal Loss (α, γ) directly on raw logits using `F.binary_cross_entropy_with_logits` for numerical stability.
`src/models.py`	`FraudMLP(nn.Module)`	Builds the feed-forward network with configurable layer sizes, `BatchNorm1d`, `Dropout`, ReLU/GELU/SiLU activations, and prior bias initialization.
`src/train_eval.py`	`train_mlp_model()`, `find_optimal_threshold()`, `compute_metrics()`	Runs the `AdamW` training loop with gradient clipping (≤ 2.0), `ReduceLROnPlateau`, PR-AUC early stopping, validation threshold selection (τ*), and metric logging.
`src/plotting.py`	`plot_eda_overview()`, `plot_pr_and_roc_curves()`, `plot_shap_explanations()`, etc.	Generates all 9 figures (`figures/fig1` to `fig9`) at 300 DPI.
`run_experiments.py`	`main()`, `objective()`	Runs the full pipeline: EDA, the 4 MLP models, the 20-trial Optuna study, XGBoost training, and SHAP analysis.
`app.py`	`load_artifacts()`, `main()`	Interactive Streamlit dashboard [43] for comparing models, testing different thresholds, and viewing local SHAP plots per transaction.

3.2 Practical Implementation Details

A few implementation choices were important for getting reliable results on this dataset:

1. **Avoiding Data Leakage:** In several public notebooks, feature scaling is applied to the entire DataFrame before calling `train_test_split`. In `src/data_utils.py`, `RobustScaler` is fitted only on `X_train[["Time", "Amount"]]` and then applied to `X_val` and `X_test`.
2. **Early Stopping on Validation PR-AUC:** Because BCE, Weighted BCE (w_+ = 24.05), and Focal Loss (γ = 2.0) have very different numerical scales, tracking validation loss is not ideal for model selection. Instead, `train_mlp_model()` tracks validation PR-AUC and saves the checkpoint from

the best epoch.

3. **Validation Threshold Calibration (τ*):** After training each model, we scan thresholds on the validation Precision-Recall curve to find $\tau^* = \text{argmax}_\tau F_1^{val}(\tau)$. We then evaluate the test set both at this calibrated threshold τ^* and at the default threshold $\tau = 0.50$.

4. Dataset

4.1 Dataset Description

We use the **ULB Credit Card Fraud Detection Dataset** [33], created by Worldline and the Machine Learning Group at Université Libre de Bruxelles [6]. It records transactions made by European cardholders over two days in September 2013. Out of **284,807 transactions**, **492 are frauds (0.1727%)** and **284,315 are normal (99.8273%)**.

Due to privacy restrictions, the original transaction features were projected using Principal Component Analysis (PCA) into 28 numerical columns named `V1` to `V28`. Only two input features were left untransformed:

- `Time`: Seconds elapsed since the first transaction in the table (0 to 172,792 seconds, covering 48 hours).
- `Amount`: Transaction value in Euros (from €0.00 to €25,691.16, with mean €88.35, median €22.00, and standard deviation €250.12).
- `Class`: Target label (1 for fraud, 0 for normal).

4.2 Exploratory Data Analysis (EDA)

Figure 1 summarizes the main patterns in the dataset:

- **Class Imbalance (Panel a):** Plotted on a logarithmic scale, normal transactions outnumber frauds by roughly 578 to 1.
- **Transaction Amounts (Panel b):** Looking at log(1 + Amount), fraudulent transactions show two peaks: one at very small values under €5 (often caused by automated "card testing" checks) and another at higher amounts between €100 and €1,000.
- **Transaction Timing (Panel c):** Normal transactions follow a clear day-night cycle, dropping sharply at night (hours 0–7 and 24–31). Fraudulent transactions do not drop off at night, making their relative share much higher during off-peak hours.
- **Most Informative PCA Features (Panel d):** Comparing the boxplots for class 0 and class 1 shows clear distribution shifts on `V14`, `V17`, `V12`, `V10`, `V4`, and `V11`.

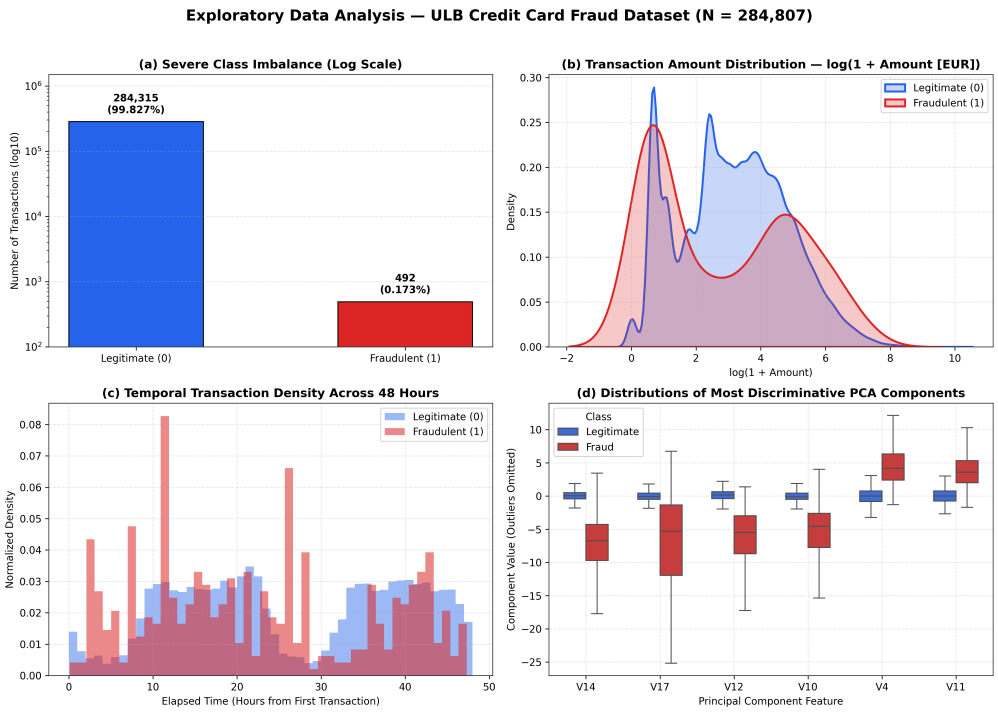


Figure 1: Four-panel Exploratory Data Analysis of the ULB Credit Card Fraud Dataset.

4.3 Data Splitting and Preprocessing

We split the dataset into Training (70%), Validation (15%), and Test (15%) sets using two-stage stratified sampling (`random_state = 42`) so that each subset has the same 0.173% fraud rate:

Dataset Split	Proportion	Total Transactions	Legitimate (`Class = 0`)	Fraudulent (`Class = 1`)	Fraud Ratio (%)
Training Set (`X_train`)	70.0%	199,364	199,020	344	0.1725%
Validation Set (`X_val`)	15.0%	42,721	42,647	74	0.1732%
Held-Out Test Set (`X_test`)	15.0%	42,722	42,648	74	0.1732%
Total Dataset	**100.0%**	**284,807**	**284,315**	**492**	**0.1727%**

Because `Amount` has heavy outliers up to €25,691.16, standardizing with the mean and standard deviation compresses most transactions into a very narrow range. Instead, we scale `Time` and `Amount` with `RobustScaler`, which subtracts the training median and divides by the training interquartile range (IQR = Q_{75} - Q_{25}):

$$\tilde{x}_j = (x_j - \text{median}_{\text{train}}(x_j)) \,/\, (Q_{75,\text{train}}(x_j) - Q_{25,\text{train}}(x_j)), \quad j \in \{\text{Time}, \text{Amount}\}$$

5. Experiments and Results

5.1 Experimental Configuration

All neural network models were trained with mini-batch size B = 1024 and fixed random seed (SEED = 42). Table 1 lists the exact hyperparameters for all five models:

Model Name	Architecture / Topology	Activation Regularization	Objective / Loss Function	Optimizer Learning Rate	Best Epoch / Iteration
1. Baseline MLP (BCE)	`30 -> 128 -> 64 -> 32 -> 1`	ReLU, `BatchNorm1d`, `Dropout(0.30)`, Prior Bias b_0=-6.36	Standard Unweighted `BCEWithLogitsLoss`	`AdamW` (lr=10^{-3}, wd=10^{-4})	Epoch 29
2. Weighted MLP (Cost-Sensitive)	`30 -> 128 -> 64 -> 32 -> 1`	ReLU, `BatchNorm1d`, `Dropout(0.30)`	Weighted BCE (w_+ = √(578.55) = 24.05)	`AdamW` (lr=10^{-3}, wd=10^{-4})	Epoch 15
3. Focal Loss MLP	`30 -> 128 -> 64 -> 32 -> 1`	ReLU, `BatchNorm1d`, `Dropout(0.30)`, Prior Bias b_0=-6.36	`BinaryFocalLoss` (α=0.75, γ=2.00)	`AdamW` (lr=10^{-3}, wd=10^{-4})	Epoch 29
4. Optuna Focal MLP	`30 -> 256 -> 128 -> 64 -> 1`	SiLU, `BatchNorm1d`, `Dropout(0.25)`, Prior Bias b_0=-6.36	`BinaryFocalLoss` (α=0.65, γ=1.75)	`AdamW` (lr=1.20 × 10^{-3}, wd=2.62 × 10^{-4})	Epoch 24
5. XGBoost (Benchmark)	`max_depth=6, subsample=0.8, colsample=0.8`	Histogram GBDT (`tree_method='hist'`)	Weighted Log-Loss (`scale_pos_weight=24.05`)	Boosting (η=0.05, aucpr early stop)	Iter 244

Figure 3 shows the validation PR-AUC and normalized training loss over epochs for the four MLP models. The **Optuna Focal MLP** reaches the highest validation PR-AUC (0.8798 at Epoch 24). By comparison, the **Weighted MLP** stops improving after Epoch 15 (0.7867) because multiplying all positive losses by 24.05 causes the model to over-amplify noisy borderline examples.

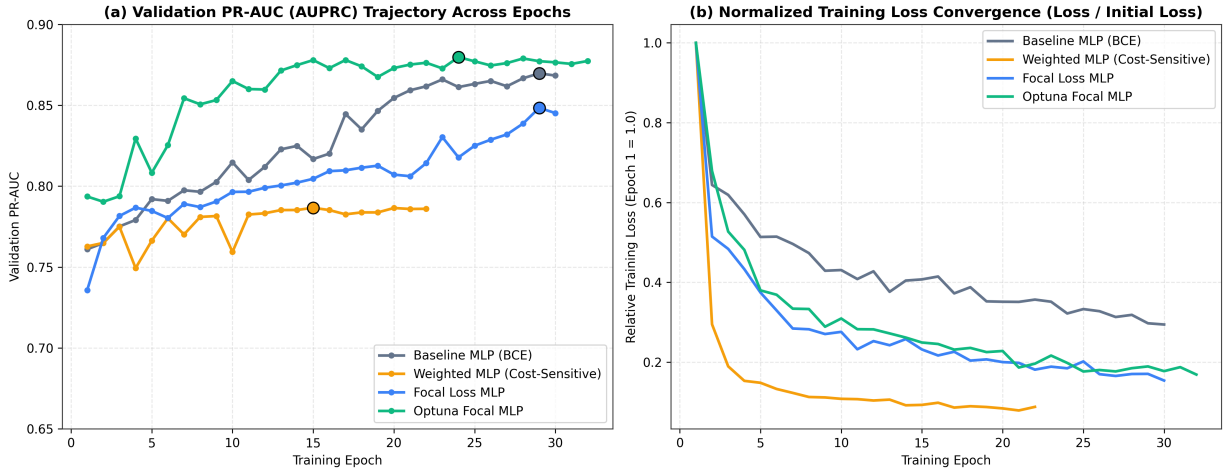


Figure 3: Validation PR-AUC trajectory and normalized training loss convergence across MLP variants.

5.2 Test Set Comparison Across Models

Table 2 reports the validation and test set metrics across all five models. The test split contains 42,722 transactions (74 frauds and 42,648 legitimate transactions). Both threshold-free ranking metrics (**PR-AUC** and **ROC-AUC**) and threshold-dependent metrics (at the validation-calibrated threshold τ* and at τ = 0.50) are included.

Model	Val PR-AUC	Val F1 (τ*)	Calibrated τ*	Test PR-AUC	Test ROC-AUC	Test F1 (τ*)	Test F1 (τ=0.5)	Test Precision (τ*)	Test Recall (τ*)	Test MCC (τ*)	Confusion Matrix (TP / FP / TN / FN)
Baseline MLP (BCE)	0.8698	0.8456	0.0875	0.8408	0.9789	0.7771	0.8000	0.7349	**0.8243**	0.7779	61 / 22 / 42,626 / 13

Model	Val PR-AUC	Val F1 (τ*)	Calibrated τ*	Test PR-AUC	Test ROC-AUC	Test F1 (τ*)	Test F1 (τ=0.5)	Test Precision (τ*)	Test Recall (τ*)	Test MCC (τ*)	Confusion Matrix (TP / FP / TN / FN)
Weighted MLP (Cost-Sensitive)	0.7867	0.8429	0.9610	0.7019	**0.9818**	0.7746	0.6563	0.8088	0.7432	0.7750	55 / 13 / 42,635 / 19
Focal Loss MLP (α=0.75, γ=2.0)	0.8485	0.8533	0.3826	0.8268	0.9706	0.7742	0.7703	0.7407	0.8108	0.7746	60 / 21 / 42,627 / 14
Optuna Focal MLP (Proposed)	**0.8798**	**0.8592**	0.5432	0.8280	0.9649	**0.8286**	0.8451	0.8788	0.7838	0.8297	58 / 8 / 42,640 / 16
XGBoost (Benchmark)	0.8775	0.8741	0.9487	**0.8422**	0.9726	0.8244	**0.8489**	**0.9474**	0.7297	**0.8312**	54 / 3 / 42,645 / 20

Figures 4, 5, and 6 show the Precision-Recall and ROC curves, the test confusion matrices, and a bar chart comparing the main metrics.

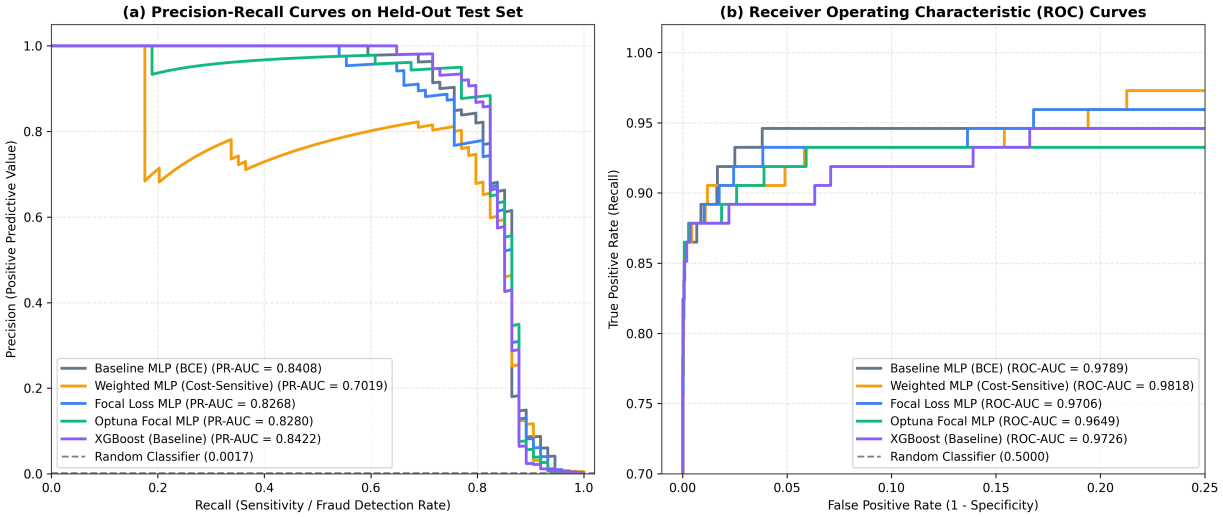


Figure 4: Precision-Recall (AUPRC) and ROC Curves on the held-out test set.

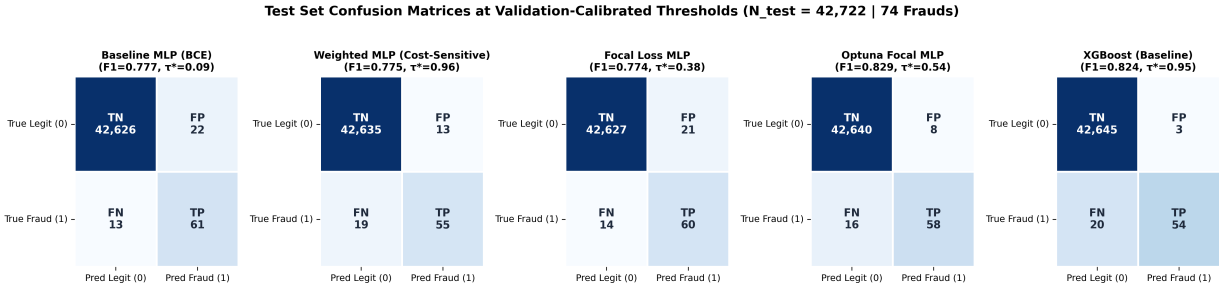


Figure 5: Confusion matrices across all 5 evaluated models on the held-out test set.

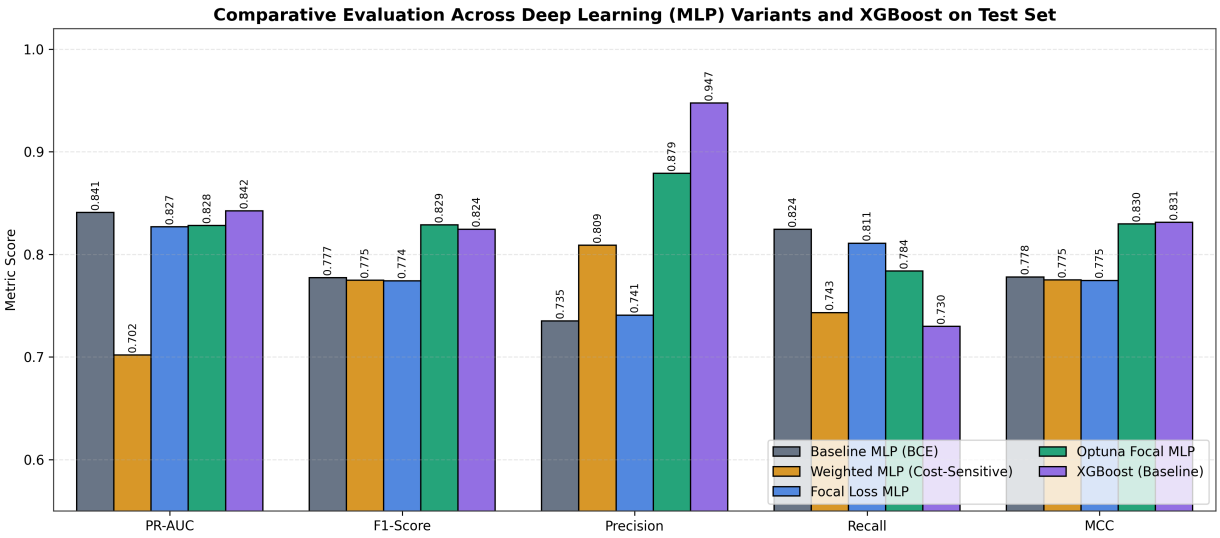


Figure 6: Comparative multi-metric bar chart across MLP variants and XGBoost on the test set.

Discussion of Results:

1. **PR-AUC vs. ROC-AUC on Imbalanced Data:** In Table 2, the **Weighted MLP** has the highest **Test ROC-AUC (0.9818)**, but at the same time the lowest **Test PR-AUC (0.7019)** and an uncalibrated F1-score of only 0.6563 at τ=0.50. Because False Positive Rate is FP / (FP + TN) and TN = 42,648, adding dozens of false positives barely changes the ROC curve. Precision (TP / (TP + FP)), however, drops immediately when false positives increase [13].

2. **Probability Calibration:** Both models that use static positive class weights (`Weighted MLP` with w_+ = 24.05 and `XGBoost` with `scale_pos_weight = 24.05`) push predicted probabilities upward, so they need high validation thresholds (τ* = 0.9610 and τ* = 0.9487) to avoid excessive false positives. In contrast, the **Optuna Focal MLP** (α=0.65, γ=1.75) keeps its probabilities well calibrated near the middle of the [0, 1] range (τ* = 0.5432), and reaches an F1-score of **0.8451** at the standard τ = 0.50 threshold.

3. **Effect of Hyperparameter Tuning:** Compared to the untuned Focal Loss MLP (128-64-32, ReLU, α=0.75, γ=2.0), the Optuna-tuned version (256-128-64, SiLU, α=0.65, γ=1.75) increases calibrated **Test F1 from 0.7742 to 0.8286** (and uncalibrated Test F1 at τ=0.50 from 0.7703 to 0.8451). Most of this gain comes from cutting False Positives from 21 down to **8**.

4. **Comparison with XGBoost:** Comparing **Optuna Focal MLP** and **XGBoost** highlights a trade-off between catching more frauds and minimizing false alarms:

- **Optuna Focal MLP** gets a slightly higher calibrated **Test F1 (0.8286 vs. 0.8244)** and catches **58 out of 74 frauds (Recall = 78.38%)**, compared to **54 out of 74 (Recall = 72.97%)** for XGBoost.
- **XGBoost** achieves a slightly higher **Test PR-AUC (0.8422 vs. 0.8280)** and higher **Precision (94.74%, with only 3 False Positives vs. 8)**, while also training much faster on CPU (6.7s vs. 228.1s).

5.3 Extension 2 Analysis: Optuna Hyperparameter Search

Across the 20 trials of the Optuna TPE search, validation PR-AUC ranged from 0.7747 (Trial #5, which used a 4-layer 256-128-64-32 network with high dropout 0.40 and a low learning rate of 3.6 × 10^{-4}) up to **0.8811 in Trial #17**.

As shown in **Figure 7**, the fANOVA hyperparameter importance scores indicate which settings mattered most:

1. `weight_decay` **(37.2% importance):** Regularizing the weights (≈ 2.62 × 10^{-4}) was the most important factor in preventing the wider 256-128-64 network from overfitting to the 344

training frauds.

2. `dropout` **(27.9% importance):** A moderate dropout rate (0.25) worked best; higher values (0.40–0.45) hurt validation PR-AUC.

3. `activation` **(13.6% importance):** Using `SiLU` improved validation PR-AUC compared to standard ReLU.

4. `lr` **(10.7%),** `architecture` **(4.6%),** `focal_gamma` **(3.7%), and** `focal_alpha` **(2.3%):** A moderate focusing parameter ($\gamma = 1.75$, $\alpha = 0.65$) worked better than very high $\gamma \geq 3.0$.

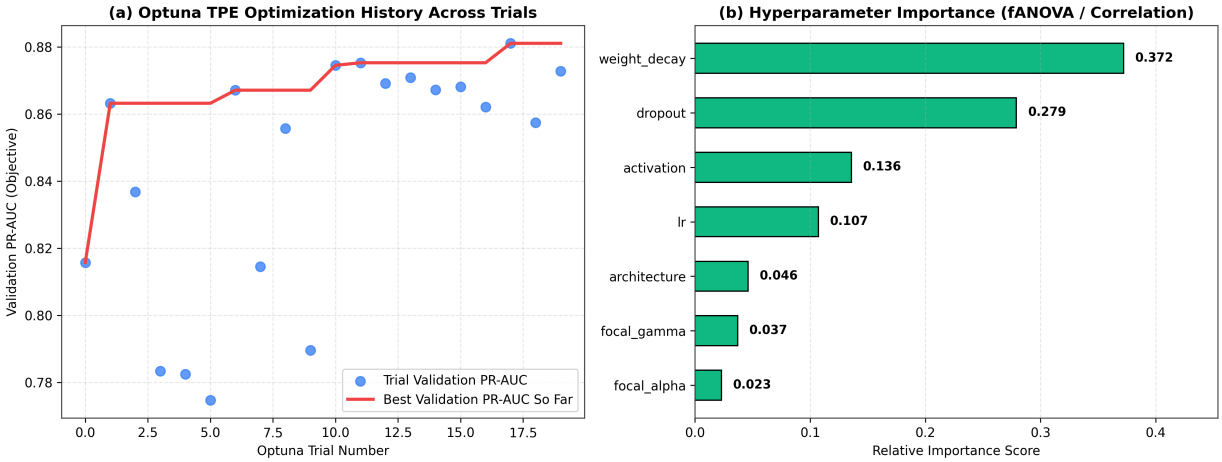


Figure 7: Optuna TPE optimization history across trials and hyperparameter importance scores.

5.4 Extension 1 Analysis: SHAP Explainability

We ran `shap.GradientExplainer` on the **Optuna Focal MLP** using 200 stratified training transactions as the background reference and computed SHAP values for 300 test transactions (all 74 test frauds plus 226 randomly sampled legitimate test transactions).

Figure 8 shows the mean absolute SHAP values (Panel a) and the Beeswarm distribution (Panel b):

- **Most Influential Features Globally:**

1. V14 (Mean |SHAP| = 1.2192; Mean Fraud SHAP = +3.0906): Has the largest impact on the model output. In the Beeswarm plot (Panel b), strongly negative values of V14 (blue points on the right side) add between +2.0 and +7.5 to the fraud log-odds.

2. V4 (Mean |SHAP| = 0.9600; Mean Fraud SHAP = +1.3810): Large positive values of V4 (red points on the right) push the prediction toward fraud.

3. `V10` (Mean |SHAP| = 0.8816; Mean Fraud SHAP = +1.7463): Negative values of `V10` strongly increase predicted fraud risk.

4. V3 (0.3736) and V12 (0.3496), followed by V1 (0.2878), V16 (0.2843), V9 (0.2760), and V17 (0.2668).

- In comparison, `Time` (Mean |SHAP| = 0.1007, rank 22) and `Amount` (Mean |SHAP| = 0.0739, rank 24) have much smaller average SHAP values, showing that the network relies primarily on the PCA components rather than simple amount thresholds.

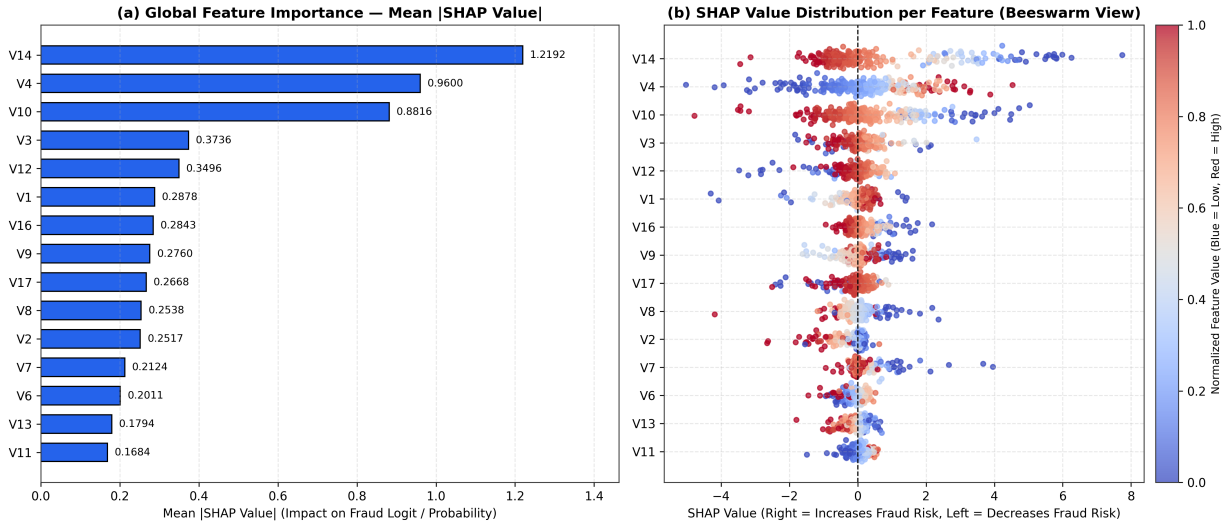


Figure 8: Global SHAP feature importance bar chart and Beeswarm impact distribution.

Figure 9 compares the local SHAP explanations for two individual test cases: a **True Positive Fraud** (left, predicted probability hat(p) = 0.9360, true class 1) and a **True Negative Legitimate Transaction** (right, predicted probability hat(p) = 0.0061, true class 0):

- For the **True Positive Fraud**, negative values on `V14 = -4.56` (SHAP = +2.3112) and `V10 = -4.16` (+1.9840), together with scaled `Amount = 10.81` (+0.6164), `V16 = -2.90` (+0.4883), `V17 = -4.67` (+0.4516), and `V4 = 1.38` (+0.4501), push the output probability to 0.9360.
- For the **Legitimate Transaction**, near-zero values on `V14 = 0.15` (SHAP = -0.4780), `V12 = 0.68` (-0.2959), and `V16 = 0.66` (-0.1892) keep the predicted fraud probability low at 0.0061.

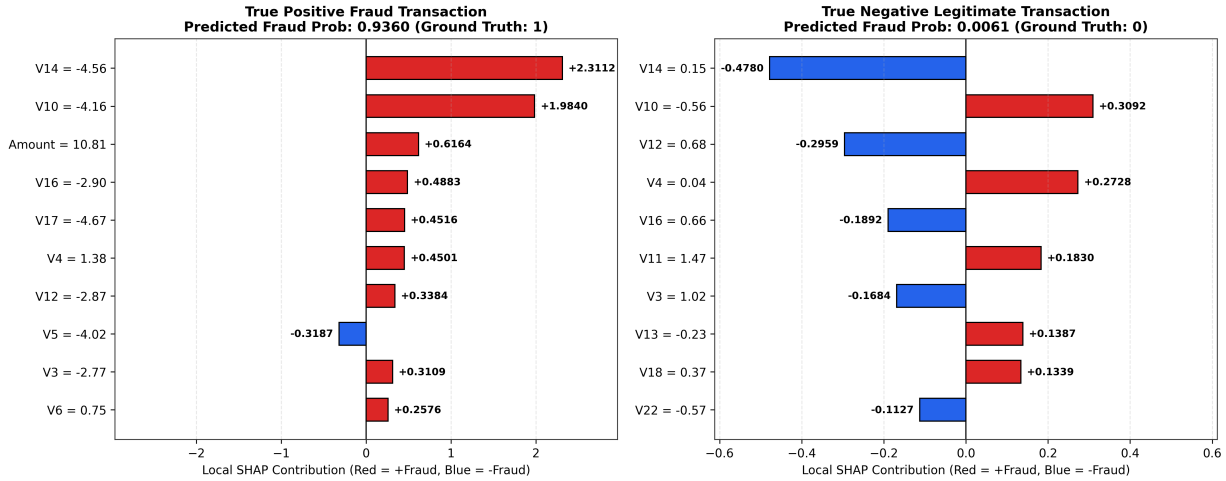


Figure 9: Local SHAP attribution plots comparing a True Positive Fraud against a Legitimate transaction.

6. Conclusion

6.1 Summary of Findings

In this project, we developed and evaluated a Multi-Layer Perceptron pipeline for credit card fraud detection on the imbalanced ULB dataset (0.1727% fraud rate). The main takeaways from our experiments are:

1. **Focal Loss Preserves Probability Calibration:** Training an MLP with Binary Focal Loss and prior bias initialization (b_0 = -6.36) handles the 1:578 class imbalance without shifting output probabilities toward 1, unlike static class weighting (w_+ = 24.05), which required an extreme threshold (τ* = 0.9610).

2. **Automated Tuning Improves Precision:** Using Optuna to tune the layer sizes (256-128-64), activation (SiLU), dropout (0.25), weight decay, and Focal Loss parameters ($\alpha = 0.65$, $\gamma = 1.75$) improved the calibrated test F1-score from 0.7742 to 0.8286 and reduced false positives from 21 to 8.
3. **MLP vs. XGBoost Trade-off:** On the test set, the Optuna Focal MLP achieved a slightly higher calibrated F1-score (0.8286 vs. 0.8244) and caught more frauds (58/74 vs. 54/74), whereas XGBoost achieved higher precision (94.74%, only 3 false positives) and a slightly higher PR-AUC (0.8422 vs. 0.8280) at a fraction of the training time.
4. **Feature Interpretability with SHAP:** SHAP analysis showed that V14, V4, V10, V3, and V12 are the primary drivers behind the network's fraud predictions.

6.2 Difficulties and Limitations

- **Small Number of Positive Test Samples:** Because the 15% validation and test splits each contain only 74 fraud cases, misclassifying 2 or 3 borderline transactions changes recall and F1 by several percentage points.
- **Anonymized PCA Features:** Because V1–V28 are PCA projections of private banking variables, we cannot engineer domain-specific features (such as spending velocity per cardholder or merchant risk scores), and SHAP plots can only point to principal components rather than human-readable attributes.
- **Short Time Horizon:** The dataset covers only 48 hours, so it does not capture seasonal changes or long-term concept drift in fraud patterns [6].

6.3 Future Work

Possible extensions of this work include combining the Optuna Focal MLP and XGBoost into a soft-voting ensemble, testing tabular transformer architectures such as FT-Transformer or TabNet, and evaluating online learning methods on longer multi-month transaction datasets to study concept drift.

7. References

[1] Abhilasha Gulhane. *Model: Build a Model for Credit Card Fraud Detection*. Medium, 2025. https://medium.com/@abhilashagulhane111/model-build-a-model-for-credit-card-fraud-detection-cd700e880a54

[5] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. *Optuna: A Next-Generation Hyperparameter Optimization Framework*. In Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery Data Mining (KDD), pp. 2623–2631, 2019.

[6] Andrea Dal Pozzolo, Olivier Caelen, Reid A. Johnson, and Gianluca Bontempi. *Calibrating Probability with Undersampling for Unbalanced Classification*. In IEEE Symposium Series on Computational Intelligence (SSCI), pp. 159–166, 2015.

[7] Tianqi Chen and Carlos Guestrin. *XGBoost: A Scalable Tree Boosting System*. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), pp. 785–794, 2016.

[8] Sergey Ioffe and Christian Szegedy. *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*. In Proceedings of the 32nd International Conference on Machine Learning (ICML), pp. 448–456, 2015.

[9] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*. Journal of Machine Learning Research (JMLR), 15(56):1929–1958, 2014.

[10] Dan Hendrycks and Kevin Gimpel. *Gaussian Error Linear Units (GELUs)*. arXiv preprint arXiv:1606.08415, 2016.

[11] Stefan Elfwing, Eiji Uchibe, and Kenji Doya. *Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning*. Neural Networks, 107:3–11, 2018.

[12] Ilya Loshchilov and Frank Hutter. *Decoupled Weight Decay Regularization*. In International Conference on Learning Representations (ICLR), 2019.

[13] Takaya Saito and Marc Rehmsmeier. *The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets*. PLOS ONE, 10(3):e0118432, 2015.

[31] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. *Focal Loss for Dense Object Detection*. In Proceedings of the IEEE International Conference on Computer Vision (ICCV), pp. 2980–2988, 2017.

[32] Scott M. Lundberg and Su-In Lee. *A Unified Approach to Interpreting Model Predictions*. In Advances in Neural Information Processing Systems 30 (NeurIPS), pp. 4765–4774, 2017.

[33] Machine Learning Group - ULB. *Credit Card Fraud Detection Dataset*. Kaggle, 2018. https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

[43] Sebastián Ramírez. *FastAPI / Web Frameworks for ML Deployment*. GitHub, 2018. (Implemented via Streamlit dashboard in `app.py`).